Homework 07

Question 1 Answers

Part a

```
library(MASS)

heart <- read.csv("data/saheart.csv")
heart$famhist <- factor(heart$famhist, levels = c("Absent", "Present"))
X <- data.matrix(heart[, names(heart) != "chd"])
y <- heart$chd

n <- nrow(X)
classes <- c(0, 1)
K <- length(classes)

pi_hat <- sapply(classes, function(k) mean(y == k))
mu_hat <- sapply(classes, function(k) colMeans(X[y == k, ]))
Sigma_hat <- Reduce(`+`, lapply(classes, function(k) {
  crossprod(scale(X[y == k, ], center = TRUE, scale = FALSE))
})) / (n - K)

names(pi_hat) <- colnames(mu_hat) <- paste0("chd = ", classes)
print(round(pi_hat, 4))
```

```
chd = 0 chd = 1
 0.6537  0.3463
```

```
print(round(mu_hat, 3))
```

```
          chd = 0 chd = 1
sbp       135.460 143.738
tobacco     2.635   5.525
ldl         4.344   5.488
adiposity  23.969  28.120
famhist     1.318   1.600
typea      52.368  54.494
obesity    25.737  26.623
alcohol    15.931  19.145
age        38.854  50.294
```

```
options(width = 85)
print(round(Sigma_hat, 2))
```

```
             sbp tobacco   ldl adiposity famhist  typea obesity alcohol    age
sbp       405.43   14.59  4.58     49.16    0.34 -15.59   18.94   64.40  95.13
tobacco    14.59   19.24  0.76      7.54    0.02  -2.06    1.83   20.52  22.77
ldl         4.58    0.76  4.00      6.03    0.09   0.34    2.66   -2.53   6.48
adiposity  49.16    7.54  6.03     56.75    0.43  -5.31   22.71   16.12  60.51
famhist     0.34    0.02  0.09      0.43    0.23   0.08    0.18    0.77   1.00
typea     -15.59   -2.06  0.34     -5.31    0.08  95.57    2.64    7.96 -20.28
obesity    18.94    1.83  2.66     22.71    0.18   2.64   17.62    4.69  15.70
alcohol    64.40   20.52 -2.53     16.12    0.77   7.96    4.69  598.28  27.89
```

```
age         95.13   22.77  6.48     60.51     1.00 -20.28   15.70   27.89 184.13
```

$\hat{\pi}_0 = 0.654$ and $\hat{\pi}_1 = 0.346$ ($n = 462$). The μ_1 class mean is higher for every predictor, most noticeably for age, tobacco, and adiposity.

Part b

```
Sigma_inv <- solve(Sigma_hat)
stopifnot(
  dim(X) == c(n, 9), length(y) == n, length(pi_hat) == K,
  dim(mu_hat) == c(9, K), dim(Sigma_inv) == c(9, 9)
)

# n x K matrix of scores x' Sigma^-1 mu_k - mu_k' Sigma^-1 mu_k / 2 + log(pi_k)
scores <- X %*% Sigma_inv %*% mu_hat -
  matrix(0.5 * diag(t(mu_hat) %*% Sigma_inv %*% mu_hat) - log(pi_hat),
         n, K, byrow = TRUE)
stopifnot(dim(scores) == c(n, K))

pred_manual <- classes[max.col(scores)]
print(table(observed = y, predicted = pred_manual))
```

```
        predicted
observed   0   1
       0 258  44
       1  73  87
```

```
print(mean(pred_manual != y))
```

```
[1] 0.2532468
```

The in-sample misclassification rate is $(44 + 73)/462 = 0.253$.

Part c

```
fit_lda <- lda(X, grouping = y)
pred_mass <- predict(fit_lda, X)$class
print(table(observed = y, predicted = pred_mass))
```

```
        predicted
observed   0   1
       0 258  44
       1  73  87
```

```
print(all(as.character(pred_mass) == pred_manual))
```

```
[1] TRUE
```

Yes. The confusion matrices are identical, and all 462 predicted labels match. `MASS::lda` uses the same empirical priors and $n - K$ pooled covariance.

Question 2 Answers

Part a

```
zip_train <- read.csv(gzfile("data/zip-train.csv.gz"), header = FALSE)
zip_test <- read.csv(gzfile("data/zip-test.csv.gz"), header = FALSE)
digit_train <- factor(zip_train[, 1])
digit_test <- factor(zip_test[, 1])
pix_train <- as.matrix(zip_train[, -1])
pix_test <- as.matrix(zip_test[, -1])

pixel_var <- apply(pix_train, 2, var)

qda_screened <- function(m) {
  sel <- order(pixel_var, decreasing = TRUE)[1:m]
  fit <- qda(pix_train[, sel], grouping = digit_train)
  pred_test <- predict(fit, pix_test[, sel])$class
  pred_train <- predict(fit, pix_train[, sel])$class
  list(
    confusion = table(observed = digit_test, predicted = pred_test),
    test_error = mean(pred_test != digit_test),
    train_error = mean(pred_train != digit_train)
  )
}

qda40 <- qda_screened(40)
print(qda40$confusion)
```

```
        predicted
observed   0   1   2   3   4   5   6   7   8   9
       0 319   0  24   6   2   2   1   0   5   0
       1   0 238   5   0  11   0   7   0   3   0
       2   8   0 168   8   8   3   0   0   3   0
       3   5   0   4 128   0  14   0   0  15   0
       4   0   2  13   1 150   1   0   1   7  25
       5  12   0   3   5   1 131   0   0   6   2
       6   0   0   8   0   1   8 148   0   5   0
       7   1   0   6   1   6   0   0 124   2   7
       8   1   0  10  12   3   2   0   0 137   1
       9   0   0   1   2   7   1   1   4   9 152
```

```
print(c(test_error = qda40$test_error, train_error = qda40$train_error))
```

```
 test_error train_error
 0.15545590  0.08339048
```

The test misclassification rate is 0.155 (training 0.083).

Part b

```
# Pixels with zero variance within each digit class
const_pixels <- sapply(levels(digit_train), function(k) {
  sum(apply(pix_train[digit_train == k, ], 2, var) == 0)
})
print(const_pixels)
```

```
 0  1  2  3  4  5  6  7  8  9
 2 99  2  0  5  0  5 23  5 18
```

Yes, the method works. QDA can now be fitted and classifies about 84.5% of test digits correctly. With all 256 pixels, some pixels are constant within a class (99 for digit 1), so the class covariance matrices are singular. The 40 highest-variance pixels sit in the central columns of the image, where every digit's strokes vary, so each 40×40 class covariance is nonsingular, and every class has at least 542 observations. QDA still estimates 820 covariance entries per class, and the gap between training and test error (0.083 vs. 0.155) shows that some of that estimation variance remains.

Part c

```
qda10 <- qda_screened(10)
print(qda10$confusion)
```

```
          predicted
observed   0   1   2   3   4   5   6   7   8   9
       0 298   0  11   1   1  16  28   4   0   0
       1   0 241  10   0   3   1   1   0   2   6
       2  19   2  95  14  14   3  22  21   5   3
       3   5   0   6 103   3  42   0   0   7   0
       4   1   5  14   0  95   2   4  15  17  47
       5  15   0   6  25   4  96   4   2   8   0
       6  41   0  15   4   0  18  78   7   5   2
       7   4   1   6   0   6   0   1 122   0   7
       8   1   2  11  36  12  11   4   1  74  14
       9   0   2   9   2  20   0   0  16  10 118
```

```
print(c(test_error = qda10$test_error, train_error = qda10$train_error))
```

```
 test_error train_error
  0.3423019   0.2888493
```

The test error more than doubles, from 0.155 to 0.342. The training error is also high (0.289), so the problem is bias, not variance. Ten pixels discard too much of the image to separate ten digits, and the large confusions (4 vs. 9, 3 vs. 5, 0 vs. 6, 3 vs. 8) are between digits that differ mainly in pixels that were dropped. The 40-predictor version estimates more parameters but keeps enough information to reduce bias by far more than it adds in variance.

Question 3 Answers

Part a

Seed: 43207.

```
gen_data <- function(m, rho) {
  y <- rep(1:2, each = m)
  X <- matrix(rnorm(2 * m * 10), ncol = 10)
  for (k in 1:2) {
    r <- if (k == 1) rho else -rho
    idx <- y == k
    # Rows of Z %*% chol(S) have covariance S
    X[idx, 1:2] <- X[idx, 1:2] %*% chol(matrix(c(1, r, r, 1), 2)) +
      matrix(c(if (k == 1) -1 else 1, 0), sum(idx), 2, byrow = TRUE)
  }
  list(X = X, y = factor(y))
```

```
}

lda_qda_error <- function(rho) {
  train <- gen_data(100, rho)
  test <- gen_data(1000, rho)
  fit_l <- lda(train$X, train$y, prior = c(0.5, 0.5))
  fit_q <- qda(train$X, train$y, prior = c(0.5, 0.5))
  c(LDA = mean(predict(fit_l, test$X)$class != test$y),
    QDA = mean(predict(fit_q, test$X)$class != test$y))
}

rhos <- c(0, 0.2, 0.4, 0.6, 0.8, 0.9)
set.seed(43207)
one_run <- sapply(rhos, lda_qda_error)
colnames(one_run) <- rhos
print(round(one_run, 4))
```

```
        0    0.2   0.4    0.6   0.8   0.9
LDA 0.157 0.1765 0.175 0.1815 0.161 0.169
QDA 0.179 0.1920 0.218 0.2195 0.127 0.103
```

Part b

```
avg_error <- sapply(rhos, function(r) rowMeans(replicate(50, lda_qda_error(r))))
colnames(avg_error) <- rhos
print(round(avg_error, 4))
```

```
         0    0.2    0.4    0.6    0.8    0.9
LDA 0.1701 0.1676 0.1704 0.1688 0.1688 0.1708
QDA 0.1969 0.1930 0.1920 0.1750 0.1340 0.1010
```

```
par(mar = c(4, 4.5, 0.5, 0.5), las = 1)
plot(
  rhos, avg_error["LDA", ], type = "b", pch = 16, lwd = 2, col = "#1F5A94",
  ylim = range(avg_error), xlab = expression("True correlation " * rho),
  ylab = "Average test error"
)
lines(rhos, avg_error["QDA", ], type = "b", pch = 17, lwd = 2, lty = 2, col = "#C84A16")
legend(
  "bottomleft", bty = "n", lwd = 2, lty = 1:2, pch = c(16, 17),
  col = c("#1F5A94", "#C84A16"), legend = c("LDA", "QDA")
)
```

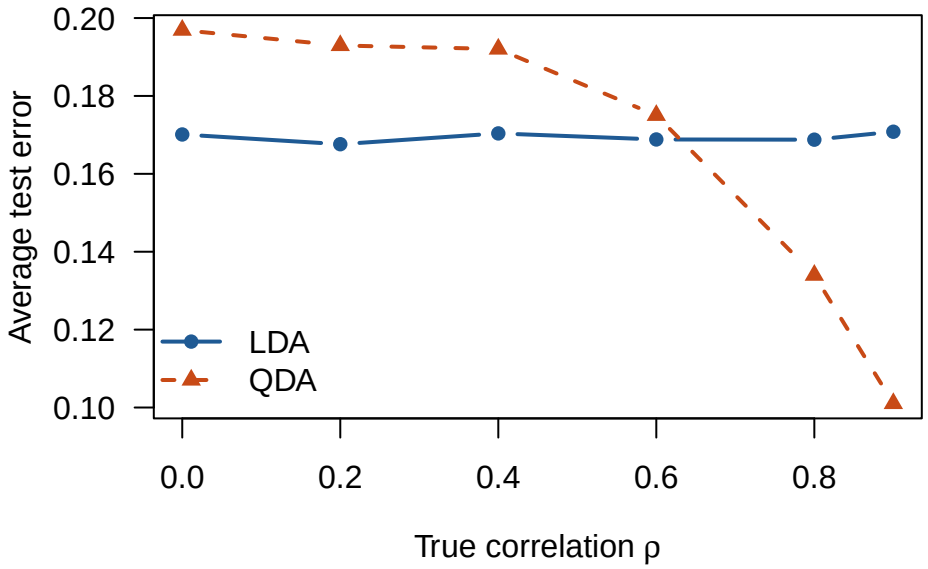


Figure 1: Average test misclassification rate over 50 repetitions.

Part c

For $\rho \leq 0.4$, LDA is better (≈ 0.170 vs. ≈ 0.19). The curves cross between $\rho = 0.6$ and 0.8, and at $\rho = 0.9$ QDA's error is 0.101 while LDA's is 0.171.

At $\rho = 0$ the class covariances are equal, so LDA's assumption is correct and it has essentially no bias. QDA estimates a separate covariance for each class (2×55 parameters vs. LDA's 55, from only 100 observations per class), which adds variance with no reduction in bias. As ρ grows, the two classes have opposite correlations. The optimal boundary becomes quadratic, and the correlation itself separates the classes. LDA's pooled covariance averages ρ and $-\rho$ to about 0, so it can use only the mean difference in X_1, and its error stays flat at about 0.17 (increasing bias). QDA captures the different covariances, and once the bias reduction outweighs its extra variance, it wins.

The eight noise covariates do not change the Bayes boundary, but both methods still estimate their means, variances, and covariances. Their sample covariances with X_1, X_2 are nonzero by chance, which tilts the fitted boundary toward noise directions. This adds variance, especially for QDA, whose per-class covariance grows from 3 to 55 parameters.

Question 4 Answers

Part a

The skill is saved as `stat432-lda-qda-analysis/SKILL.md` (full content in Part c). It takes the dataset, response, and predictors as inputs. Its candidates are LDA, QDA, a probability blend of the two, and PCA + LDA/QDA, and it specifies the CV procedure, the selection rule, and a one-page report with a figure, a table, and the test confusion matrix.

Part b

```
set.seed(43274)
test_id <- unlist(lapply(split(seq_along(y), y), function(i) {
  sample(i, round(0.3 * length(i)))
}))
```

```r
X_tr <- X[-test_id, ]; y_tr <- y[-test_id]
X_te <- X[test_id, ]; y_te <- y[test_id]

n_folds <- 10
folds <- integer(length(y_tr))
for (k in classes) {
  i <- which(y_tr == k)
  folds[i] <- sample(rep(1:n_folds, length.out = length(i)))
}

w_grid <- c(0, 0.25, 0.5, 0.75, 1)
d_grid <- 1:ncol(X)
candidates <- data.frame(
  family = rep(c("Blend", "PCA + LDA", "PCA + QDA"), c(5, 9, 9)),
  setting = c(w_grid, d_grid, d_grid),
  n_par = c(2 * 9 + 45, rep(2 * 9 + 2 * 45, 4), 2 * d_grid + d_grid * (d_grid + 1) / 2,
            2 * d_grid + d_grid * (d_grid + 1))
)

# P(chd = 1) for every candidate; PCA and scaling use Xa only
candidate_prob <- function(Xa, ya, Xb) {
  p_lda <- predict(lda(Xa, ya), Xb)$posterior[, 2]
  p_qda <- predict(qda(Xa, ya), Xb)$posterior[, 2]
  center <- colMeans(Xa); spread <- apply(Xa, 2, sd)
  pc <- prcomp(scale(Xa, center, spread))
  Za <- pc$x
  Zb <- scale(Xb, center, spread) %*% pc$rotation
  pca_prob <- function(method) sapply(d_grid, function(d) {
    predict(method(Za[, 1:d, drop = FALSE], ya), Zb[, 1:d, drop = FALSE])$posterior[, 2]
  })
  cbind(sapply(w_grid, function(w) w * p_qda + (1 - w) * p_lda),
        pca_prob(lda), pca_prob(qda))
}

cv_fold_error <- sapply(1:n_folds, function(k) {
  p <- candidate_prob(X_tr[folds != k, ], y_tr[folds != k], X_tr[folds == k, ])
  colMeans((p > 0.5) != y_tr[folds == k])
})
candidates$cv <- rowMeans(cv_fold_error)
candidates$se <- apply(cv_fold_error, 1, sd) / sqrt(n_folds)

# One-SE rule: fewest parameters among candidates within 1 SE of the minimum
best <- which.min(candidates$cv)
within_1se <- which(candidates$cv <= candidates$cv[best] + candidates$se[best])
selected <- within_1se[which.min(candidates$n_par[within_1se])]

p_test <- candidate_prob(X_tr, y_tr, X_te)
candidates$test <- colMeans((p_test > 0.5) != y_te)
selected_confusion <- table(observed = y_te,
                            predicted = as.integer(p_test[, selected] > 0.5))
```

Analysis report: LDA/QDA classification of coronary heart disease

Data and setup. The data have 462 men (34.6% with `chd = 1`) and nine numerical predictors (`famhist` coded 1/2). With seed `43274`, a stratified 30% test set (139) was reserved before any modeling, leaving 323 training observations. All candidates were compared on the same stratified 10-fold CV folds within the training set, with scaling and PCA estimated within each fold's training part.

Candidate models. Ordinary LDA and QDA; a blend $w\,\hat{P}_{\text{QDA}} + (1-w)\hat{P}_{\text{LDA}}$ for $w \in \{0, 0.25, 0.5, 0.75, 1\}$; and PCA + LDA and PCA + QDA on the first $d = 1, \ldots, 9$ standardized principal components. Classification uses empirical priors and a 0.5 cutoff.

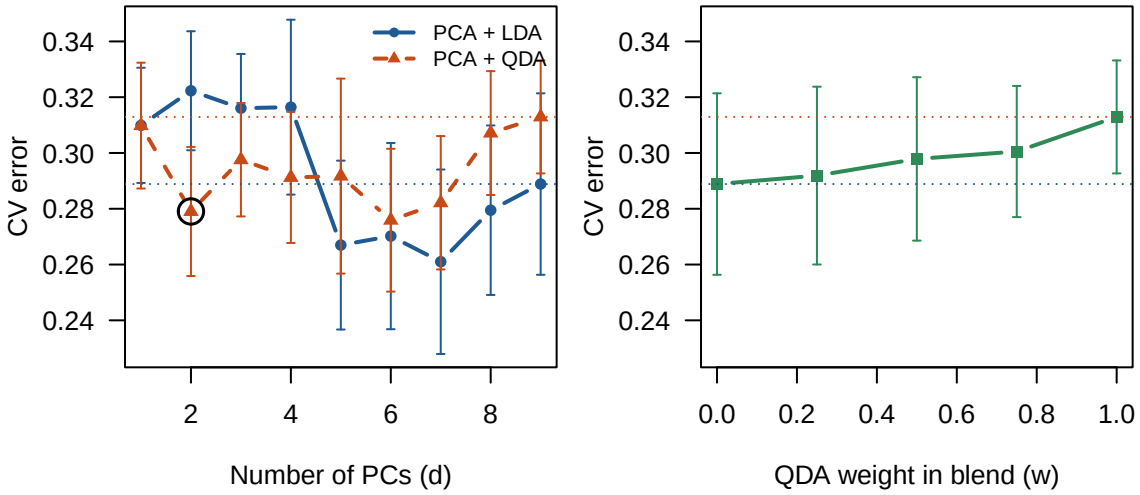


Figure 2: 10-fold CV misclassification rate (±1 SE). Dotted lines: ordinary LDA (blue) and QDA (orange). Circled point: selected model.

Model	Setting	CV error	CV SE	Test error
LDA	–	0.289	0.033	0.259
QDA	–	0.313	0.020	0.273
Blend (best CV)	w = 0	0.289	0.033	0.259
PCA + LDA (best CV)	d = 7	0.261	0.033	0.245
PCA + QDA (best CV)	d = 6	0.276	0.026	0.230
PCA + QDA (selected)	**d = 2**	**0.279**	**0.023**	**0.317**

Selected model. The minimum CV error was PCA + LDA with $d = 7$ (0.261, SE 0.033). Thirteen of the 23 candidates were within one SE, and among those the one with the fewest parameters was **PCA + QDA with** $d = 2$ (CV error 0.279). Refit on all 323 training observations, its test error is 0.317. Test confusion matrix (rows observed, columns predicted): `chd = 0`: 74 / 17, `chd = 1`: 27 / 21.

Findings and limitations. All candidates have CV errors between 0.26 and 0.32, and those differences are about one SE, so CV cannot clearly separate the candidates. Moving the blend weight toward QDA increases CV error, so unequal class covariances do not help here and the extra QDA parameters mainly add variance. Reducing to 5–7 principal components gives the lowest CV errors. The selected two-component model is simple but has low sensitivity (21 of 48 cases detected). Its test error is worse than its CV estimate (0.317 vs. 0.279), and some non-selected models did better on the test set (not used for selection). With only 139 test observations (SE ≈ 0.04), one split, imbalanced classes, a 0.5 cutoff, and the normality assumption (`famhist` is binary, `alcohol` and `tobacco` are skewed), all of these estimates are imprecise.

Part c

`SKILL.md` is 3,829 bytes. I enabled explicit-only invocation in two ways. The file sits in the homework folder rather than an automatically loaded skills folder (`~/.claude/skills/` or `.claude/skills/`), and the frontmatter sets `disable-model-invocation: true`, so Claude Code never loads it unless I name it. Prompt used:

> Use the skill in `stat432-lda-qda-analysis/SKILL.md` to analyze `data/saheart.csv`. The response is `chd`; the predictors are the nine other variables, coding `famhist` as 1 for Absent and 2 for Present. Seek the best predictive performance among the models the skill allows, in R, for my Quarto PDF report.

I reviewed the CV results. The one-SE rule admitted 13 of 23 candidates because the fold SEs are large (≈ 0.03), so it chose a simpler model than the CV minimum. I kept the rule rather than revise it, because changing it after this review would make the selection depend on more than training and CV results.

```
---
name: stat432-lda-qda-analysis
description: >-
  Run an LDA/QDA classification analysis on a user-supplied dataset and
  write a one-page statistical report. Use only when the user explicitly
  invokes this skill by name and supplies the data file, response, and
  predictors.
disable-model-invocation: true
---

# LDA/QDA Classification Analysis

The user supplies the dataset, the response, the predictors, and any coding
rules. Do not assume anything else about the data. Use R (`MASS`) or Python
(`scikit-learn`), whichever the user's report uses.

## Candidate models

Only classifiers built from LDA, QDA, or their combination are allowed.

1. Ordinary LDA (always included as a comparison).
2. Ordinary QDA (always included as a comparison).
3. Blend: predicted probability `w * P_QDA + (1 - w) * P_LDA`, with
   `w` on a grid such as 0, 0.25, 0.5, 0.75, 1.
4. PCA + LDA and PCA + QDA: standardize the predictors, keep the first `d`
   principal components, then fit the classifier. Use a small grid of `d`
   from 1 to the number of predictors.

Use empirical class priors and classify to the class with the largest
posterior probability (cutoff 0.5 for two classes) unless the user says
otherwise. Skip a candidate if any class covariance is singular and say so.

## Procedure

1. Set and report one random seed.
2. Reserve a stratified test set (default 30%) before looking at any model
   results. Do not use it again until step 6.
3. Create one set of stratified K-fold CV folds (default K = 10) within the
   training set. Use the same folds for every candidate and every setting.
4. Inside each fold, estimate all preprocessing (standardization, PCA) from
```

```
   the fold's training part only, then apply it to the held-out part.
5. Compute the CV misclassification rate and its standard error across folds
   for every candidate. Select the candidate with the lowest CV error; if
   several are within one standard error, prefer the simplest (fewest
   parameters) and say so.
6. Refit the selected model and its preprocessing on the full training set
   and evaluate it once on the test set. Also report test error for ordinary
   LDA and QDA as comparisons, clearly labeled as not used for selection.
7. Never revise candidates, grids, or the selection after seeing test results.

## Report (at most one printed page)

Write in plain, concise sentences. No code in the report body. Use these
sections, in order:

1. **Data and setup** (2-3 sentences): sample size, class proportions,
   number of predictors, split sizes, K, seed.
2. **Candidate models** (1-2 sentences): which families and grids were tried.
3. **Figure**: CV misclassification rate (y-axis) against the tuning
   parameter (x-axis), one line per model family, with +/- 1 SE bars and
   dashed horizontal reference lines for ordinary LDA and QDA. Mark the
   selected model. Use a figure about 6 in by 2.8 in, colorblind-safe colors
   plus distinct line types and point shapes, labeled axes, and a legend
   without a box.
4. **Table**: one row per family's best setting plus ordinary LDA and QDA,
   with columns Model, Setting, CV error, CV SE, Test error. Round errors to
   3 decimals and bold the selected model.
5. **Selected model**: name, setting, CV error, test error, and the test
   confusion matrix (observed in rows, predicted in columns) in one sentence
   or a small inline table.
6. **Findings and limitations** (3-5 sentences): what drove the result
   (e.g. whether unequal covariances help), how large the differences are
   relative to their standard errors, and limitations such as test-set size,
   the single split, the normality assumption, and class imbalance.

Check before finishing: the report fits on one page, every number in the
text matches the table, and the test set was used only once for selection
purposes.
```